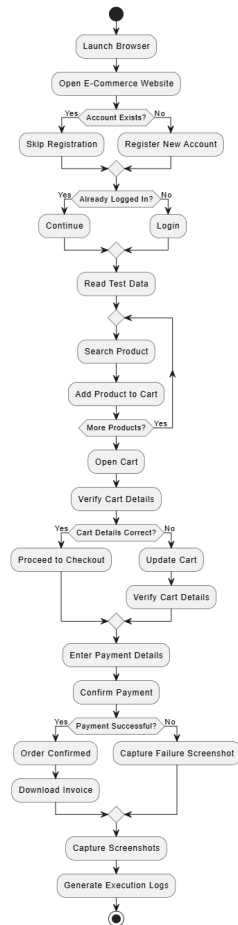


E-Commerce Website Automation

Selenium WebDriver Capstone Project Report

Overall E-Commerce Automation Workflow



Overall Automation Workflow

Author: Sayan Dey

September 23, 2026

2. Abstract

This report presents an end-to-end e-commerce website automation project built with Python, Selenium WebDriver, Firefox, and Pytest. The automated scenario opens the Automation Exercise website, registers or reuses a test account, signs in when needed, searches a data-driven product list, adds products to the cart, proceeds through checkout, enters test payment data, and records the final confirmation state. The project separates account, search, cart, logging, and screenshot responsibilities into focused modules. Explicit waits, exception logging, CSV test data, and screenshot evidence are used to make the workflow repeatable and easier to diagnose.

Contents

2. Abstract	1
3 Introduction	3
4 Problem Statement	3
5 Objectives	3
6 Technologies Used	4
7 System Requirements	4
8 Project Architecture	4
9 Project Workflow	4
10 Project Structure	5
11 Test Automation Framework	5
12 Selenium WebDriver	5
13 Web Element Locators	6
14 Page Object Model (POM)	6
15 Test Case Design	6
16 Test Execution	7
17 Synchronization and Wait Handling	7
18 Exception Handling	7
19 Assertions and Validation	8
20 Test Results	8

21 Challenges Faced	12
22 Advantages	12
23 Limitations	12
24 Future Scope	12
25 Conclusion	12

3 Introduction

Modern e-commerce systems require reliable validation of complete customer journeys, not only isolated page actions. A simple failure in login, product search, cart handling, or checkout can prevent a customer from completing a purchase. This project automates that business-critical journey on the Automation Exercise demo website.

The project is a browser-based UI automation exercise. It uses a single Firefox session to maintain authentication and cart state from the landing page through payment confirmation. The main Pytest scenario is implemented in `test_ecommerce.py`; `launch.py` provides a standalone runner for the same high-level sequence. The output of a successful run includes logs and screenshots that show the major application states.

4 Problem Statement

Manual validation of an e-commerce purchase flow is repetitive, slow, and susceptible to human oversight. It requires repeated account setup, product discovery, form completion, cart navigation, and payment submission. Changes in page timing or dynamic content can also make manual checks inconsistent.

The problem addressed by this project is how to create a repeatable automated test that validates the main online-shopping workflow from account access to order completion. The solution must handle existing accounts, dynamic page elements, multiple products, and evidence collection while remaining understandable and maintainable.

5 Objectives

- Automate registration, login, product search, cart addition, checkout, payment, and order completion.
- Read test inputs from configuration and CSV files instead of hard-coding product and account data in the test flow.
- Use Selenium explicit waits to synchronize browser actions with page readiness.
- Capture visual evidence at important stages and write execution details to a log file.
- Organize code into reusable modules for account, search, cart, and utility tasks.

6 Technologies Used

Technology	Use in the Project
Python	Primary implementation language for the automation scripts.
Selenium WebDriver	Browser automation API used to control Firefox and interact with the web application.
Pytest	Test runner and fixture framework for the end-to-end scenario.
Mozilla Firefox	Browser used by the WebDriver session.
CSV / ConfigParser	External test data and application URL configuration.
Python logging	Timestamped execution and failure messages in <code>log.txt</code> .
LaTeX and MiKTeX	Report preparation and PDF compilation.

Table 1: Technologies used in the project.

7 System Requirements

The project requires a Windows system with Python 3.10 or later, Mozilla Firefox, Selenium 4, Pytest, network access to <https://automationexercise.com/>, and a compatible Firefox WebDriver environment. Modern Selenium versions can manage the driver automatically when the local browser environment is supported; otherwise GeckoDriver must be available on the system path. To compile this report, MiKTeX with `pdflatex` is required.

The test uses the source files `config.ini`, `testdata.csv`, and `products.csv`. The browser needs permission to create or overwrite PNG files in the `screenshots` folder and append runtime information to `log.txt`.

8 Project Architecture

The architecture is a modular functional design. The test runner owns the browser lifetime and orchestrates the business sequence. Domain modules receive the shared Selenium `driver` instance, allowing the same session, authenticated user, and cart contents to persist throughout the run. The full workflow diagram is presented on the title page.

The data layer contains the application URL in `config.ini`, customer details in `testdata.csv`, and products in `products.csv`. The support layer contains screenshot capture and logging. This separation makes it possible to update input data without rewriting the full workflow.

9 Project Workflow

The workflow begins by starting Firefox and opening the configured URL. Registration reads the first test-data record and attempts to create an account. If the account-information form is unavailable after signup, the email is treated as already registered

and registration is skipped. Login then checks whether a Logout link exists; if not, it signs in with CSV credentials and waits for the logged-in indicator.

The test reads every product row, searches for the exact product name, adds the matching item to the cart, and continues shopping until all products have been processed. It then opens the cart, proceeds to checkout, places the order, submits test card values, waits for a page transition from the payment URL, and captures the order-success screenshot. Finally, the browser is closed by fixture teardown or the standalone runner's `finally` block.

10 Project Structure

File or Folder	Purpose
<code>test_ecommerce.py</code>	Pytest fixture and complete end-to-end test scenario.
<code>launch.py</code>	Standalone runner for the same business workflow.
<code>register.py</code> , <code>login.py</code>	Account creation and authentication.
<code>search.py</code>	Products-page navigation, product search, and result check.
<code>cart.py</code>	Add-to-cart, cart view, checkout, order placement, and payment.
<code>screenshot.py</code> , <code>logger.py</code>	Screenshot creation and centralized logging.
<code>popup.py</code>	Optional advertisement-iframe removal helper.
<code>products.csv</code> , <code>testdata.csv</code>	Product and customer test data.
<code>screenshots/</code>	Evidence images produced during execution.
<code>Report/</code>	Flowchart source, flowchart image, and this report.

Table 2: Project file structure.

11 Test Automation Framework

Pytest is used as the test automation framework. The `driver` fixture creates Firefox, configures it with eager page loading and image loading disabled, opens the target website, and yields the ready driver to `test_ecommerce_flow`. After the test completes or fails, the fixture closes the browser.

The standalone `launch.py` runner performs similar setup through helper functions and uses `try/except/finally` to ensure driver cleanup. The Pytest runner is preferable for repeatable automated execution because fixtures provide a clear setup and teardown boundary.

12 Selenium WebDriver

Selenium WebDriver controls Firefox through `webdriver.Firefox`. The driver opens URLs, locates page elements, fills forms, selects dropdown values, clicks controls, executes JavaScript scrolling, reads the current URL, and saves browser screenshots.

The cart module demonstrates several WebDriver capabilities: it locates a product card, scrolls the Add to Cart link to the center of the viewport, clicks it, waits for the

confirmation modal, and continues shopping. Registration uses Selenium's **Select** helper for date-of-birth and country dropdowns. Payment automation enters the site's test card data and waits until the browser leaves the payment page.

13 Web Element Locators

The project uses a combination of locator strategies selected according to the structure of the target page:

- **CSS selectors:** stable `data-qa` fields such as login email, signup button, payment inputs, and checkout links.
- **IDs:** direct identifiers such as `search_product`, `submit_search`, newsletter, and gender radio buttons.
- **Partial link text:** navigation controls including Products, Cart, Signup / Login, and Logout.
- **XPath:** exact product-name verification, parent product-card selection, modal controls, and text-based validation.

Using exact product text in the search result prevents the script from adding an unrelated result. The locator choices are encapsulated inside the modules, reducing their impact on the orchestration code.

14 Page Object Model (POM)

The project does not currently implement a formal class-based Page Object Model. Instead, it uses a functional modular approach: `register.py`, `login.py`, `search.py`, and `cart.py` group actions by page or business area. This gives the project several POM-like benefits, including separation of locators from the central workflow and reusable page-level functions.

For a larger project, the next step would be to convert these modules into page classes such as `LoginPage`, `ProductsPage`, `CartPage`, and `PaymentPage`. Each class would store locators and expose business-focused methods. This would reduce duplication and improve maintainability if the website UI changes.

15 Test Case Design

The main test case is an end-to-end positive purchase scenario. Preconditions are an available Automation Exercise website, Firefox, valid test data, and network access. The input products are Blue Top, Men Tshirt, and Sleeveless Dress. The expected outcome is that each product can be found and added, checkout can be reached, payment can be submitted, and navigation reaches an order-completion URL.

The scenario also includes decision paths for account state. A new email follows the registration route, while an existing email skips the rest of registration and moves to login. A pre-authenticated browser session bypasses login through the Logout-link check.

16 Test Execution

The test can be run with `python -m pytest -v test_ecommerce.py`; the standalone flow can be executed with `python launch.py`. A recorded successful execution in `log.txt` processed three products, reached a `/payment_done/` URL, logged a successful completion message, and closed Firefox.

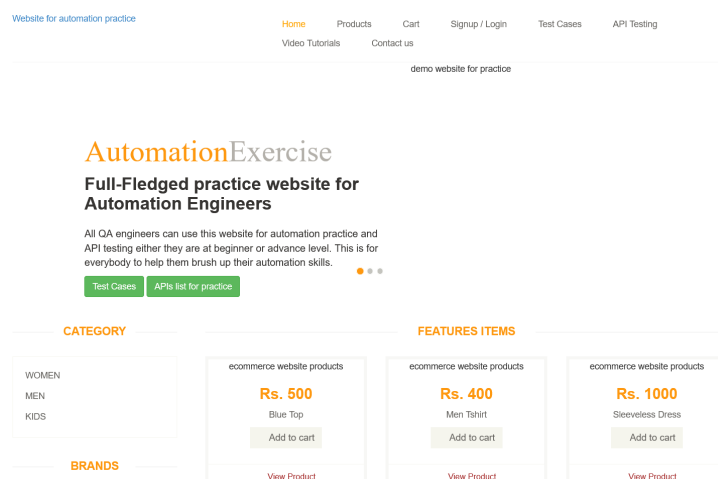


Figure 1: Homepage captured after browser initialization.

17 Synchronization and Wait Handling

The automation primarily uses `WebDriverWait` with Selenium expected conditions rather than relying on arbitrary sleep delays. It waits for elements to become visible or clickable before interacting with navigation links, forms, search controls, modal buttons, and check-out actions. It also waits for the cart URL and for navigation away from the payment page.

This approach is more reliable than immediately attempting actions while the page is still changing. The one fixed delay in the Pytest test is only used to keep the browser visible briefly after the workflow; it is not used to decide whether a page element is ready.

18 Exception Handling

Each workflow module surrounds its main operations with `try/except` blocks. When an operation fails, the code logs a descriptive message with the error and re-raises the exception so Pytest or the standalone runner reports a failed test. The standalone runner uses a `finally` block to call `driver.quit()` even after failure.

This design preserves the original failure signal while adding context to `log.txt`. A future improvement is to capture a failure screenshot automatically from every exception handler before re-raising the error.

19 Assertions and Validation

The project currently validates progress through explicit conditions rather than Pytest `assert` statements. Examples include waiting for the “Logged in as” text, locating an exact matching product, confirming that the cart URL contains `/view_cart`, confirming that the Account Created heading is visible for new accounts, and waiting until the payment URL changes after submission.

These checks provide meaningful validation points; however, the test can be strengthened by adding explicit assertions for product name, cart quantities, item totals, checkout address details, and an order-success message. The current `quantity` column in `products.csv` is not applied by the automation.

20 Test Results

The available log and screenshots show a successful run. The account already existed, so registration was skipped; login succeeded; all three configured products were searched and added; the cart, checkout, and payment stages completed; and the final URL changed to an order-completion page. The following evidence images were produced by the test.

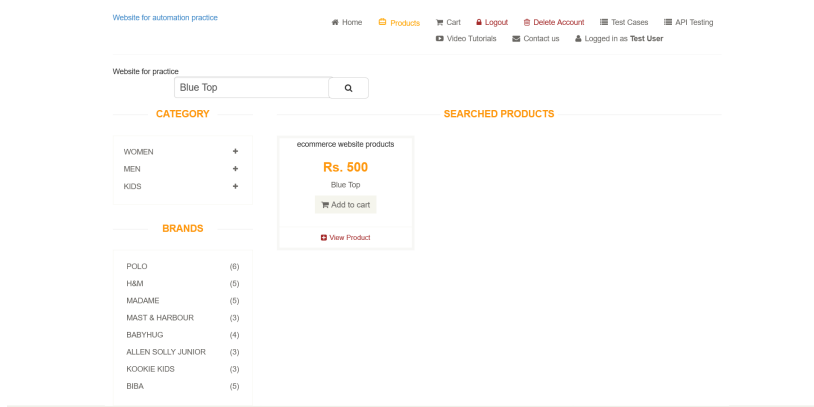


Figure 2: Blue Top located before cart addition.

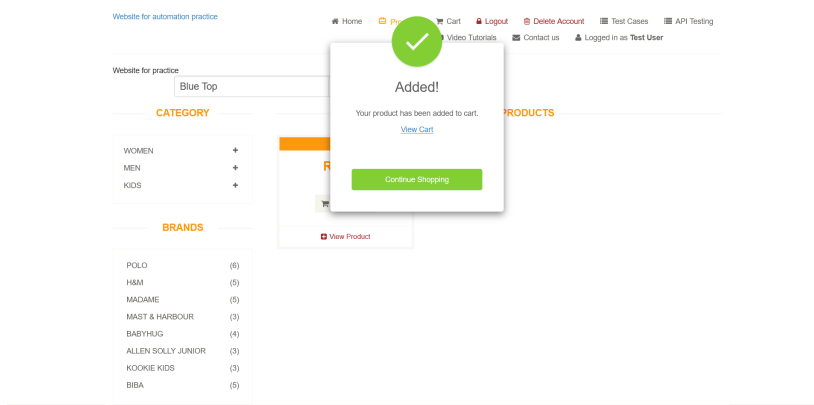


Figure 3: Blue Top added-to-cart confirmation.

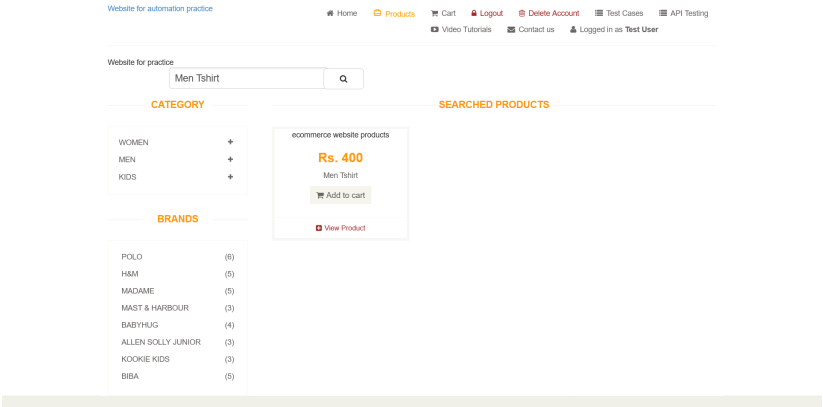


Figure 4: Men Tshirt located before cart addition.

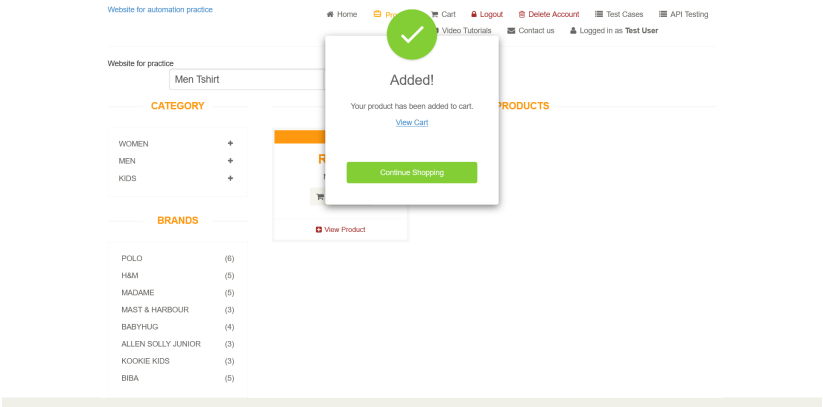


Figure 5: Men Tshirt added-to-cart confirmation.

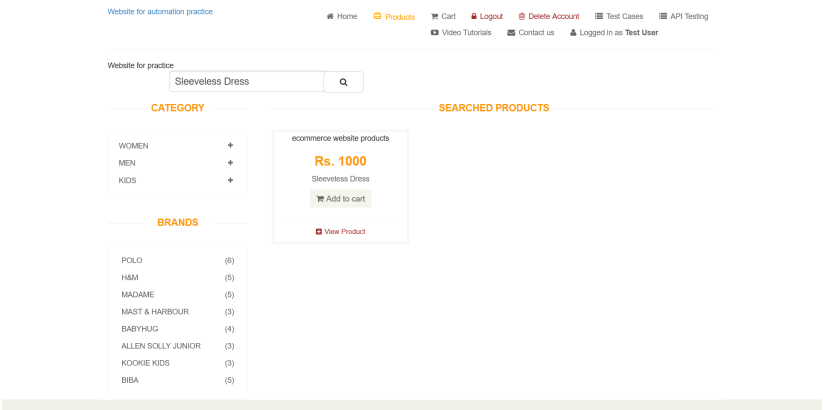


Figure 6: Sleeveless Dress located before cart addition.

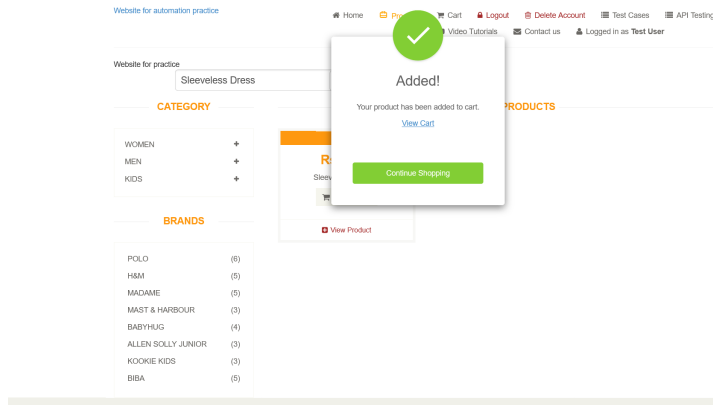


Figure 7: Sleeveless Dress added-to-cart confirmation.

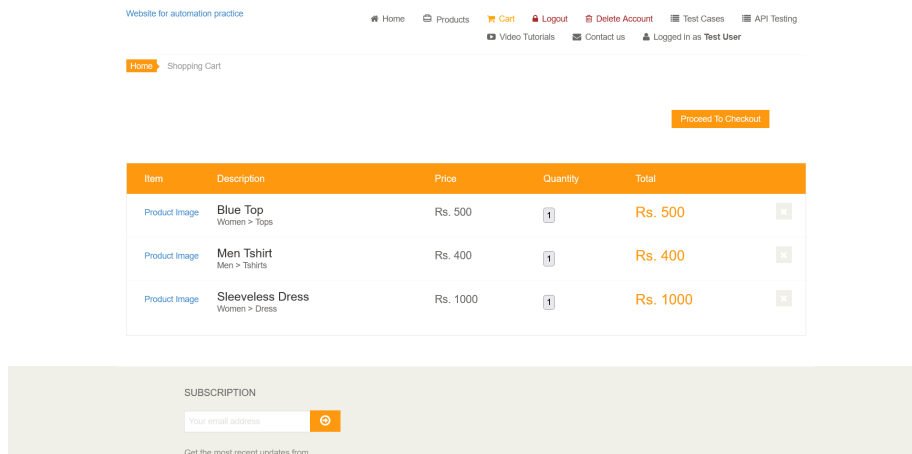


Figure 8: Cart view after products were added.

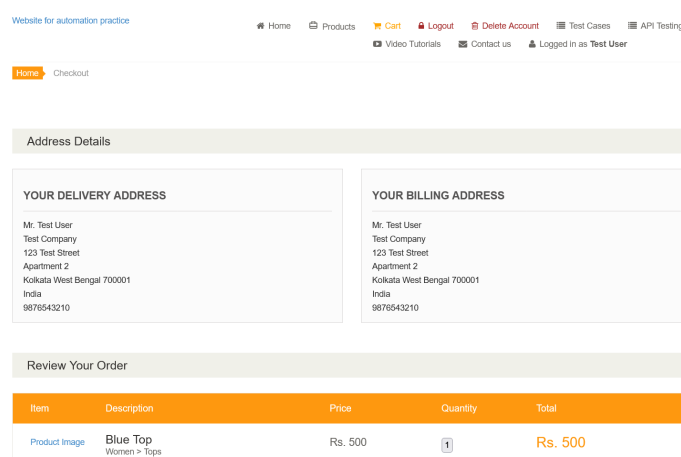


Figure 9: Checkout page.

Website for automation practice

Home Products Cart Logout Delete Account Test Cases API Testing
Video Tutorials Contact us Logged in as Test User

Home Payment

Payment

Name on Card

Card Number

CVC Expiration
ex. 311 MM YYYY

Pay and Confirm Order

SUBSCRIPTION

Your email address

Get the most recent updates from our site and be updated your self...

Figure 10: Payment page.

Website for automation practice

Home Products Cart Logout Delete Account Test Cases API Testing
Video Tutorials Contact us Logged in as Test User

Home Payment

Payment

Name on Card

Card Number

CVC Expiration

Pay and Confirm Order

SUBSCRIPTION

Your email address

Get the most recent updates from our site and be updated your self...

Figure 11: Payment details entered.

Automation Exercise website

Home Products Cart Logout Delete Account Test Cases API Testing
Video Tutorials Contact us Logged in as Test User

ORDER PLACED!

Congratulations! Your order has been confirmed!

Download Invoice Continue

SUBSCRIPTION

Your email address

Get the most recent updates from our site and be updated your self...

Figure 12: Order confirmation page.

21 Challenges Faced

The main challenge was advertisement overlays. Dynamic advertisement iframes can cover controls, intercept mouse clicks, and cause Selenium interactions to fail even when the target control exists. This is particularly relevant around signup, search, and product-card actions.

The project includes `remove_ads(driver)` in `popup.py`. It runs JavaScript that removes iframe elements with common Google Ads and DoubleClick identifiers, names, or source URLs. The calls are currently commented out in the primary flow, so normal site behavior is preserved; they can be enabled before sensitive interactions if overlays recur. Other challenges include the timing of dynamic elements, reuse of an existing account, and reliance on the external demo website's changing page structure.

22 Advantages

- Automates a complete customer journey rather than isolated page actions.
- Uses reusable modules and external CSV/configuration data.
- Applies explicit waits to improve reliability on dynamic pages.
- Produces screenshots and logs that support troubleshooting and reporting.
- Supports both a Pytest-based test and a standalone execution route.

23 Limitations

The project is tied to one demo website and its current UI locators. It does not yet use a formal Page Object Model, parameterized tests, test markers, reporting plugins, or a dependency manifest. Cart quantity values are present in the CSV file but are not applied. The flow does not validate cart totals, download invoices, or capture screenshots automatically on every failure. Test credentials and payment data are stored in plain text for demo purposes and must not be used for real accounts or real payment systems.

24 Future Scope

Future development can introduce page-object classes, a `requirements.txt` file, environment variables for sensitive data, parameterized positive and negative test cases, headless execution, HTML test reports, and CI/CD integration. The cart flow can be extended to update quantities from `products.csv`, validate prices and totals, handle coupon cases, verify final confirmation text, and download invoices. Failure handling can capture a screenshot and browser diagnostic data automatically for each failed action.

25 Conclusion

This project demonstrates a practical Selenium-based e-commerce automation solution. It uses Firefox and a shared WebDriver session to execute registration or login, product

search, cart addition, checkout, payment, and final confirmation. CSV configuration, explicit waits, logging, and screenshots make the scenario repeatable and provide clear test evidence.

The implementation is a strong foundation for broader automated testing. Its next stage should focus on stronger assertions, cart validation, a formal Page Object Model, safer test-data handling, and CI-ready execution. The current workflow successfully illustrates how browser automation can validate a complete online purchase journey efficiently and consistently.